Lambda vs Kappa

two pipelines, or one log you can replay

Watch the 4-min video

In one line: Lambda runs a batch layer that recomputes accurate results from all raw data plus a speed layer for the latest hours, and merges them at query time, at the cost of two codebases; **Kappa** uses one streaming pipeline over a long-retention log and fixes mistakes by **replaying** the log into a new version of the output.

1 Real-time numbers that change overnight

The dashboard shows today's sales updating every second. The next morning finance reruns the numbers and they are slightly different, because late refunds and duplicate events were cleaned up overnight. You want numbers that are **fresh** (stream processing, lesson 32) and **correct and correctable** (batch processing, lesson 31). Lambda and Kappa are two ways of getting both.

2 Lambda architecture

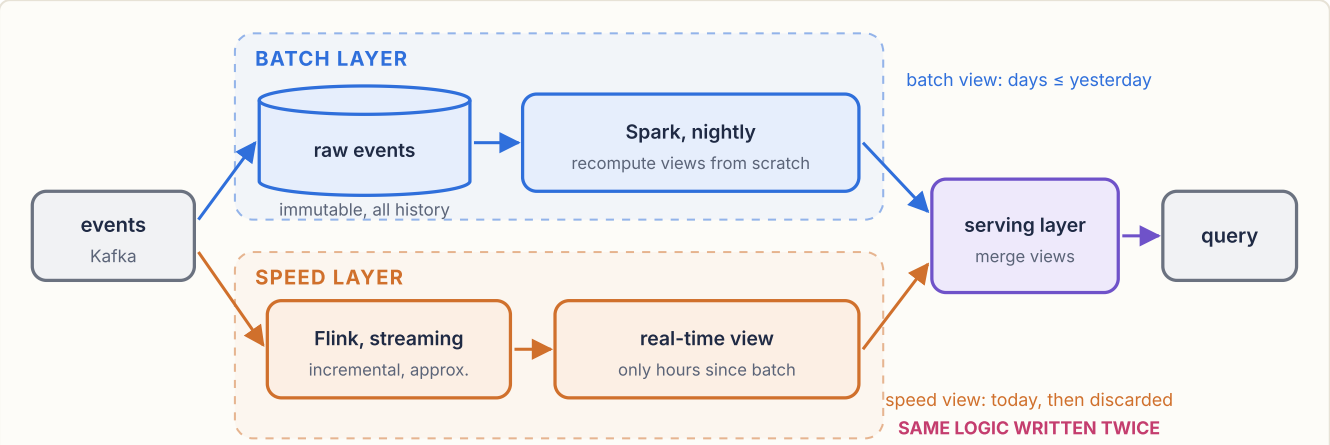


Figure 1. Lambda (Nathan Marz, around 2011). Every event goes to both layers; the serving layer hides the seam between them.

- **Batch layer:** keeps the **master dataset**, every raw event, immutable and append-only (e.g. Parquet on S3). It periodically recomputes complete views *from scratch*, so any earlier mistake disappears on the next run.
- **Speed layer:** processes the same events as a stream and maintains views for only the data the last batch run hasn't covered. These are allowed to be approximate and are thrown away once the batch catches up.
- **Serving layer:** answers queries by merging the batch view (up to the batch cut-off) with the real-time view (after it).

The strength is **robustness**: a bug, a bad deploy or corrupted stream state is fixed by correcting the code and letting the next batch recompute from raw data. Human error is survivable because raw data is never mutated.

The weakness is **duplication**. The same business logic (dedup rules, currency conversion, refund handling) exists twice, in a batch engine and a stream engine with different APIs and semantics. The two

drift, so the numbers jump at the seam between today and yesterday, and every change is built, tested and deployed twice. Jay Kreps' 2014 essay "Questioning the Lambda Architecture" proposed Kappa as the answer.

3 Kappa architecture

Remove the batch layer: **everything is a stream**. The event log (usually Kafka) is the master dataset, kept for as long as you may need to reprocess. One streaming job, one codebase, computes all results. Batch becomes a special case: a stream that starts at an old offset and runs fast.

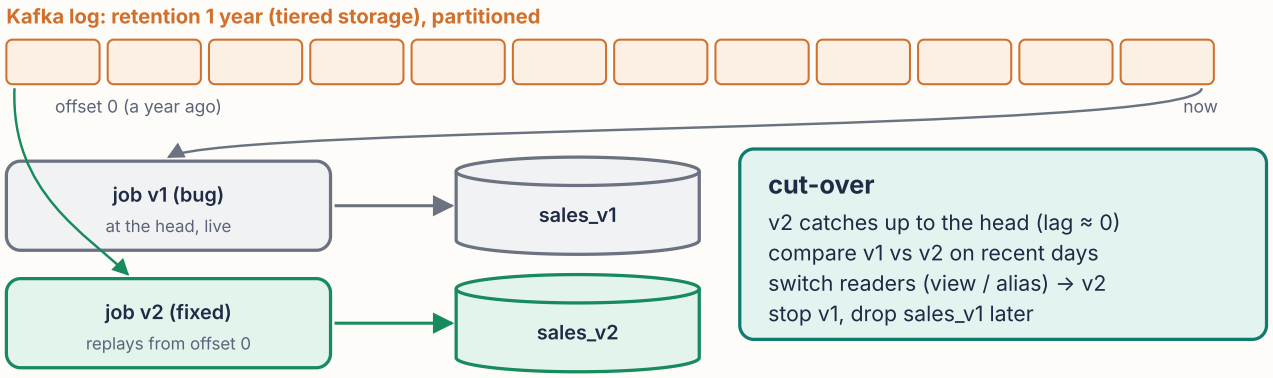


Figure 2. Correcting a mistake in Kappa: run the fixed job as a new version from the start of the log into a new table, then switch readers.

Reprocessing by replay. Deploy job v2 with a new consumer group, starting from offset 0 (or from the first day affected), writing to `sales_v2`. It processes history as fast as the cluster allows. When it has caught up with the head of the log, compare the two outputs, switch readers (a view, alias or config flag), stop v1 and later drop `sales_v1`. Because the job uses **event time**, replayed data lands in the same windows it did originally.

4 What Kappa requires

- **Enough retention:** months or years of raw events in the log. Kafka tiered storage (KIP-405) and similar features in other logs move old segments to object storage, making this affordable; or archive the log to a lakehouse table and replay from there.
- **Replay speed:** if live traffic averages 50,000 events/s and a replay runs at 1 million events/s (20× live), 90 days of history takes about 4.5 days. Plan parallelism (partitions) and sink capacity for this.
- **A capable stream engine:** event time, watermarks and late data, large state, and exactly-once output (lesson 32).
- **Versioned outputs:** writing into a new table per version, so a bad replay never corrupts what readers see.

	Lambda	Kappa
Pipelines / codebases	two (batch + stream)	one (stream)
Source of truth	raw files in the data lake	the event log
Fixing a bug	fix both; next batch run recomputes	fix once; replay into a new version
Consistency	batch and speed results may disagree	one logic, one answer
Good at	heavy batch work: model training, huge joins, ad-hoc recomputation	event-driven metrics, features, alerts
Main cost	maintaining two systems	long retention and fast replay

Which today? Usually Kappa-style, helped by engines that run the same code in batch and streaming modes (Flink, Spark Structured Streaming, Apache Beam) and by lakehouse tables (lesson 34) that can be written by streams and recomputed by batch. Lambda still makes sense when some computations simply are batch-shaped, such as training models on a year of data or giant joins that don't fit a stream.

WORKED EXAMPLE: REAL-TIME AND CORRECTED DAILY NUMBERS

Lambda: a Flink job updates `today_sales` every few seconds. At 02:00 a Spark job recomputes yesterday from raw events in S3, removing duplicate events and applying refunds that arrived late, and writes `daily_sales`. The dashboard reads `daily_sales` for past days and `today_sales` for today, and the Flink numbers for yesterday are discarded. **Kappa:** one Flink job, keyed by order, dedupes by event ID and applies refunds by event time with allowed lateness, writing upserts to `sales_v1`. A currency bug is found: `job v2` replays the Kafka log from the bug's start date into `sales_v2`, catches up in a few hours, and the dashboard switches to v2.

TRADE-OFFS

- Lambda is robust to bugs and well-suited to batch-heavy work, but doubles code, tests and operations.
- Kappa has one truth and one codebase, but depends on long log retention and replay that is fast enough.
- Replays cost compute and sink capacity; running v1 and v2 side by side doubles the load for a while.
- Unified engines reduce Lambda's duplication but do not remove the two operational paths.

COMMON MISTAKES

- Choosing Kappa with 7-day log retention, then finding a bug that started a month ago.
- Replaying into the live output table, so readers see half-old, half-new numbers.
- Using processing time in the stream job, so a replay puts a year of data into today.
- In Lambda, letting batch and stream logic drift without comparing their outputs.

5 Key terms

Batch layer: recomputes complete views from all raw data

Speed layer: streams recent data for fresh, approximate views

Serving layer: merges batch and real-time views at query time

Replay: re-reading the log from an old offset with a new job

Master dataset: immutable raw events, the source of truth

Tiered storage: old log segments kept cheaply in object storage

6 Interview questions

Q1. Lambda versus Kappa: which and why? BASIC

Lambda runs a batch layer for accurate recomputation and a speed layer for fresh results, at the cost of two codebases; Kappa uses one streaming pipeline and fixes mistakes by replaying the log.

Q2. Which would you choose today? BASIC

Usually Kappa, if the log keeps enough history and the engine handles late data; Lambda when heavy batch jobs don't fit streaming.

Q3. In Kappa, how do you deploy a fix to a metric's logic without downtime? SCENARIO

Run the fixed job as v2 from the start offset into a new table, wait until it catches up, validate against v1, then switch readers and retire v1.

Q4. Live traffic is 20k events/s; you must reprocess 60 days within 2 days. What replay rate do you need? SCENARIO

60 days of data in 2 days is 30× live speed: about 600k events/s, which sets partition count, parallelism and sink capacity.

Q5. Why do Lambda dashboards often 'jump' at midnight? TRICKY

Yesterday switches from the speed layer's approximate numbers to the batch layer's recomputed ones, and the two implementations differ.

Q6. Is Kappa possible without Kafka keeping years of data? TRICKY

Yes, if raw events are archived to a lakehouse table and the same streaming code can read that table as a bounded stream for replays.

Next: SD-Q-009 · Data Pipelines backendroadmap.com